

CSS Module 2: Layout and Modern CSS

Flexbox, alignment, positioning, z-index, centring techniques, grid, units and CSS functions.

11 topics

Contents

- 09 Understanding Flexbox
- 10 flex-wrap and gap
- 11 justify and align Properties
- 12 Understanding the position Property
- 13 Absolute and Relative Positioning with Offsets
- 14 position: fixed and position: sticky
- 15 z-index and Stacking
- 16 Centring a Box with position and transform
- 17 CSS Grid: Rows and Columns
- 18 CSS Units
- 19 CSS Functions: clamp, min, max and var

09 Understanding Flexbox

The two axes · Container and item properties · flex shorthand

Turning on flex layout

```
.container { display: flex; }
```

The moment this is declared, the element becomes a **flex container** and all of its direct children become **flex items**. Grandchildren are unaffected, and the children now sit in a row by default instead of stacking as blocks.

The two axes

flex-direction	Main axis	Cross axis
row, the default	Left to right	Top to bottom
row-reverse	Right to left	Top to bottom
column	Top to bottom	Left to right
column-reverse	Bottom to top	Left to right

THE SINGLE MOST USEFUL IDEA IN FLEXBOX

`justify-content` always works along the main axis and `align-items` always works along the cross axis. Change `flex-direction` to column and the two swap their visual meaning. Learning the axes rather than memorising the visual result is what makes flexbox feel simple.

Properties on the container

- `flex-direction`, `flex-wrap` and the `flex-flow` shorthand.
- `justify-content`, `align-items` and `align-content`.
- `gap`, for spacing between items.

Properties on the items

Property	Meaning
<code>flex-grow</code>	The share of leftover free space this item receives. The default is 0, meaning it does not grow.
<code>flex-shrink</code>	How readily the item shrinks when space runs short. The default is 1.
<code>flex-basis</code>	The starting size before growing or shrinking is applied. The default is <code>auto</code> , that is, the content size.
<code>flex</code>	The shorthand for the three above, and the recommended way to write them.
<code>align-self</code>	Overrides <code>align-items</code> for one item only.
<code>order</code>	Changes the visual order. The DOM order stays the same, so heavy use harms keyboard navigation.

```
.item { flex: 1; } /* grow 1, shrink 1, basis 0 -> equal columns */
.item { flex: 0 0 200px; } /* fixed 200px, never grows or shrinks */
.item { flex: 1 1 300px; } /* aim for 300px, then share the free space */
```

The difference between `flex: 1` and `flex-grow: 1` matters: the shorthand also sets the basis to zero, so all items end up truly equal, whereas `flex-grow: 1` alone keeps the content size as the starting point and wider content stays wider.

A layout this solves immediately

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}  
/* logo on the left, links on the right, everything vertically centred */
```

QUICK RECAP

- `display: flex` affects only the direct children.
- The main axis follows `flex-direction`; the cross axis is perpendicular to it.
- Prefer the flex shorthand, and remember that `flex: 1` sets the basis to zero.

10 flex-wrap and gap

Multi-line flex containers · Spacing without margins

The default is a single line

A flex container does not wrap. When the items are too wide, they shrink instead, and once they can shrink no further they overflow. This is the behaviour of `flex-wrap: nowrap`, which applies unless it is changed.

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

Value	Behaviour
<code>nowrap</code>	The default. Everything stays on one line and items shrink or overflow.
<code>wrap</code>	Items move onto new lines as required.
<code>wrap-reverse</code>	Wraps in the opposite cross-axis direction, so new lines appear above rather than below.

The shorthand `flex-flow: row wrap` sets the direction and the wrapping together.

A responsive card grid without media queries

```
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}  
  
.card {  
  flex: 1 1 280px; /* aim for 280px, grow to fill, wrap when it cannot */  
}
```

Each card tries to be two hundred and eighty pixels wide. When three no longer fit, one moves to the next line and the remaining cards expand to fill the row. The layout adapts without a single breakpoint.

The gap property

```
gap: 20px; /* both row and column gaps */  
gap: 12px 24px; /* row gap | column gap */  
row-gap: 12px;  
column-gap: 24px;
```

- `gap` creates space **between** items only. It never adds space at the outer edges, which is exactly what margins used to get wrong.
- There is no need for the old pattern of a margin on every item followed by a negative margin on the container, or a `:last-child` rule to remove the final margin.
- Gaps do not collapse the way margins do, so the spacing is exactly the value declared.
- The same property works in grid layouts and in multi-column text, so it is worth learning once.

TWO POINTS THAT CATCH PEOPLE OUT

When wrapping is enabled and there are several lines, `align-items` aligns items within each line, whereas `align-content` distributes the lines themselves. With `nowrap` there is only one line, so `align-content` has no effect at all. Also note that a percentage gap is resolved against the size of the container, so fixed units are safer for consistent spacing.

QUICK RECAP

- Flex containers do not wrap unless `flex-wrap` is set.
- `flex: 1 1 <basis>` with `wrap` produces a responsive grid without breakpoints.
- `gap` spaces items internally and never adds outer edge spacing.

11 justify and align Properties

justify-content · align-items · align-content and align-self

The rule that never changes

`justify-content` distributes items along the **main axis**. The align family works along the **cross axis**. Which direction those axes point is decided by `flex-direction`.

justify-content

Value	Result along the main axis
<code>flex-start</code>	The default. Items sit at the start.
<code>flex-end</code>	Items sit at the end.
<code>center</code>	Items are grouped in the middle.
<code>space-between</code>	The first and last items touch the edges, and the free space is divided equally between the items.
<code>space-around</code>	Each item receives equal space on both sides, so the outer gaps appear half the size of the inner ones.
<code>space-evenly</code>	Every gap, including the outer ones, is exactly equal.

align-items

Value	Result along the cross axis
<code>stretch</code>	The default. Items expand to fill the container on the cross axis, unless they have a fixed size on that axis.
<code>flex-start</code> , <code>flex-end</code>	Items align to the start or the end.
<code>center</code>	Items are centred on the cross axis.
<code>baseline</code>	The text baselines of the items line up, which is ideal when labels use different font sizes.

align-content

This property distributes **the lines** of a wrapped container along the cross axis. It therefore requires `flex-wrap: wrap` and more than one line; with a single line it does nothing. It accepts the same values as `justify-content`, along with `stretch`.

align-self

```
.container { display: flex; align-items: center; }
.logo      { align-self: flex-start; } /* this one item ignores the container rule */
```

Perfect centring

```
.center {
  display: flex;
  justify-content: center; /* horizontal, because the direction is row */
  align-items: center;    /* vertical */
  min-height: 100vh;
}
```

CHANGE THE DIRECTION AND THE MEANING SWAPS

With `flex-direction: column`, `justify-content: center` centres vertically and `align-items: center` centres horizontally. Nothing about the properties has changed; only the axes have rotated. Also note that a vertical `justify-content` can only do its work when the container has a height to distribute.

Where stretch quietly helps

Cards of unequal text length end up the same height inside a flex row purely because `align-items` defaults to `stretch`. Setting `align-items:`

`center` switches that off, and the cards then size to their own content, which is a frequent surprise.

QUICK RECAP

- `justify-content` works on the main axis, `align-items` on the cross axis.
- `align-content` needs wrapping and multiple lines to have any effect.
- `align-self` overrides the container setting for a single item.

12 Understanding the position Property

static, relative, absolute, fixed and sticky · The document flow

What the document flow means

By default every element is laid out in the normal flow: block elements stack downwards and inline elements sit beside one another. The `position` property decides whether an element stays in that flow, and whether the offset properties `top`, `right`, `bottom` and `left` have any effect on it.

Value	Stays in the flow	Offsets are measured from
<code>static</code>	Yes, this is the default	Nowhere. Offsets and z-index are ignored entirely.
<code>relative</code>	Yes, the original space is preserved	Its own original position.
<code>absolute</code>	No, it is removed from the flow	The nearest positioned ancestor, or the page if none exists.
<code>fixed</code>	No	The viewport, so it does not move when the page scrolls.
<code>sticky</code>	Yes, until the threshold is reached	The nearest scrolling ancestor, switching between relative and fixed behaviour.

The most important consequence

Removing an element from the flow means the surrounding content behaves as though it no longer exists. Neighbouring elements move up to fill the space, and the parent no longer counts that element when calculating its own height. A container holding only absolutely positioned children will therefore collapse to zero height.

Positioned versus not positioned

In the specification, an element is described as **positioned** when its `position` is anything other than `static`. This single definition explains two behaviours that otherwise seem unrelated:

- An absolutely positioned child looks upwards for the nearest **positioned** ancestor to use as its reference.
- `z-index` only applies to **positioned** elements, along with flex and grid items.

Choosing the right value

- `relative` is most often used not to move an element at all, but simply to make it the reference point for an absolutely positioned child.
- `absolute` suits badges, close buttons, dropdown panels and overlays that must sit on top of a specific parent.
- `fixed` suits a persistent header, a floating action button or a modal backdrop.
- `sticky` suits section headings and table headers that should follow the user only while their section is visible.

POSITION IS NOT A LAYOUT SYSTEM

Absolute positioning takes an element out of the flow, so it cannot respond to the size of its siblings. Use flexbox or grid for arranging a layout, and reserve positioning for elements that genuinely need to sit on top of something else.

QUICK RECAP

- Offsets are ignored on static elements.
- Absolute and fixed elements leave the flow; relative and sticky keep their space.
- Anything other than static counts as positioned, which unlocks both the containing block and z-index.

13 Absolute and Relative Positioning with Offsets

The containing block · top, right, bottom and left · The inset shorthand

How relative behaves

```
.box { position: relative; top: 20px; left: 30px; }
```

The box moves twenty pixels down and thirty pixels to the right of where it would otherwise have been, but the **space it originally occupied is retained**. Neighbouring elements do not move, which means a relatively positioned element can visually overlap its neighbours while leaving a gap behind it.

How absolute behaves

```
.parent { position: relative; }          /* the reference point */

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

The badge leaves the flow and is placed ten pixels from the top and right edges of `.parent`. The pairing of a relative parent with an absolute child is the standard pattern for badges, overlays, close buttons and dropdowns.

THE CONTAINING BLOCK RULE

An absolutely positioned element searches **upwards** for the nearest ancestor whose position is not static. If no such ancestor exists, it positions itself against the initial containing block, which effectively means the page. This is why a badge sometimes flies to the top corner of the whole document: the parent was never made relative.

Offsets on both sides

```
.overlay {
  position: absolute;
  top: 0; right: 0; bottom: 0; left: 0;
}
```

```
.overlay { position: absolute; inset: 0; } /* the modern shorthand */
```

When opposite offsets are both given and no width is declared, the element **stretches** between them. This is how a full-cover overlay is created without stating any dimensions. If a width is also declared, the width wins and the `right` value is ignored in a left-to-right document.

Behaviour worth knowing

- An absolutely positioned element shrinks to fit its content unless a width or a pair of opposite offsets is supplied.
- Percentage offsets are calculated against the containing block, not the element itself.
- Negative values are permitted, which is how a badge is made to hang slightly outside its card.
- Setting `overflow: hidden` on the parent will clip an absolutely positioned child that extends beyond it.
- A transform on an ancestor also creates a containing block for absolute children, which is a subtle cause of unexpected positioning.

THE COLLAPSING PARENT

A container whose children are all absolutely positioned has no in-flow content, so its height becomes zero and the following section slides underneath it. Give the container an explicit height, or keep at least one child in the normal flow.

QUICK RECAP

- Relative moves the element visually but keeps its original space.
- Absolute is measured from the nearest positioned ancestor, so mark that ancestor relative.
- Opposite offsets without a width cause the element to stretch; `inset: 0` is the shorthand.

14 position: fixed and position: sticky

Viewport-anchored elements · Scroll thresholds · Why sticky silently fails

Fixed

```
.navbar {
  position: fixed;
  top: 0; left: 0; right: 0;
  height: 64px;
}

body { padding-top: 64px; } /* replace the space the navbar no longer occupies */
```

A fixed element is positioned against the **viewport** and therefore stays in place while the page scrolls. Because it leaves the normal flow, the content beneath it moves up and slides under the bar, which is why compensating padding is added to the body.

THE EXCEPTION THAT SURPRISES EVERYONE

If any ancestor has a `transform`, `filter`, `perspective` or `will-change` value other than none, that ancestor becomes the containing block and the fixed child scrolls along with it. A fixed header that mysteriously scrolls away is almost always caused by a transform on a parent, frequently one added by an animation.

Sticky

```
.section-title {
  position: sticky;
  top: 0;
  background: #ffffff;
}
```

A sticky element behaves as `relative` while it is in view, and switches to behaving like `fixed` once the scroll position reaches the threshold given by `top`. It never leaves the flow, so it keeps its space and the layout does not jump.

It is also confined to its parent. When the parent scrolls out of view, the sticky element leaves with it, which is exactly the behaviour wanted for section headings and grouped table headers.

Why sticky often does not work

Cause	Fix
No threshold has been declared.	A value for <code>top</code> , <code>bottom</code> , <code>left</code> or <code>right</code> is compulsory.
An ancestor has <code>overflow: hidden</code> , <code>auto</code> or <code>scroll</code> .	Remove it, or move the sticky element outside that container.
The parent is not tall enough.	Sticky only travels within its parent, so a short parent means almost no travel.
The parent is a flex container with default alignment.	The stretched item may leave no room to move; set <code>align-self: flex-start</code> .

Choosing between them

- Use **fixed** when the element must remain visible for the entire page, such as a global header, a chat button or a modal backdrop.
- Use **sticky** when the element should follow the user only within its own section, such as a category heading, a table header or a sidebar filter panel.
- On mobile devices, avoid fixing large elements to the bottom, since browser toolbars change the visible viewport height. The `dvh` unit helps here.

QUICK RECAP

- Fixed is anchored to the viewport and leaves the document flow.
- Sticky keeps its space and activates at a declared threshold.
- Ancestor overflow and ancestor transforms are the two usual culprits when either one misbehaves.

15 z-index and Stacking

Painting order · Stacking contexts · Why a large value can still lose

The prerequisite

`z-index` affects only elements that are **positioned**, meaning their position is relative, absolute, fixed or sticky, along with flex and grid items. On a static element the property is ignored completely, and this is by far the most common reason for it appearing not to work.

```
.modal {  
  position: fixed; /* without this line, z-index does nothing */  
  z-index: 100;  
}
```

The default painting order

When no `z-index` is set, the browser paints in a defined order: backgrounds and borders of the root, then non-positioned block boxes, then floats, then inline content, and finally positioned elements in the order in which they appear in the markup. This is why a later element naturally covers an earlier one.

Stacking contexts

A stacking context is a self-contained layer. Inside it, children are stacked among themselves, and the entire context is then placed as a single unit within its parent context. The consequence is important: **a child can never escape its parent's layer**, no matter how large its `z-index` is.

```
.parent-a { position: relative; z-index: 1; }  
.child    { position: absolute; z-index: 9999; }  
  
.parent-b { position: relative; z-index: 2; }  
/* parent-b covers the child, because 2 is greater than 1 at the level that matters */
```

A new stacking context is created by, among other things: a positioned element with a `z-index` other than auto, any `opacity` below 1, any `transform`, `filter` or `will-change`, and every flex or grid item that has a `z-index`.

HOW TO DEBUG THIS

When an element refuses to come to the front, do not raise its `z-index`. Walk up the tree instead and find the nearest ancestor that creates a stacking context. The competition is happening at that level, and the fix belongs there.

Practical guidance

- Use a small, documented scale such as dropdown 10, sticky header 100, modal 1000, toast 1100. Values like 99999 signal that the real problem was never diagnosed.
- Negative values are allowed. An element with `z-index: -1` is painted behind its parent's content but still in front of the parent's background.
- Remember that an opacity below one silently creates a context, which explains why a faded card sometimes traps its own dropdown.

QUICK RECAP

- `z-index` requires a positioned element, or a flex or grid item.
- Children are trapped inside their parent's stacking context.
- opacity and transform create stacking contexts without any `z-index` being written.

16 Centring a Box with position and transform

The fifty per cent technique · Why transform completes it · Modern alternatives

The classic technique

```
.parent { position: relative; }

.child {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

Why the transform is necessary

The offsets `top` and `left` position the **top-left corner** of the child at the centre of the parent, not the centre of the child. The box therefore appears pushed down and to the right by half its own size.

`translate(-50%, -50%)` then moves the element back by half of **its own width and height**. This is the essential detail: percentages inside `translate` refer to the element itself, whereas percentages in `top` and `left` refer to the containing block. Because the element measures itself, the technique works even when the size of the box is unknown or changes with its content.

When the size is known

```
.child {
  position: absolute;
  width: 200px; height: 100px;
  top: 50%; left: 50%;
  margin-top: -50px; /* half the height */
  margin-left: -100px; /* half the width */
}
```

This older approach works only while the dimensions are fixed, which is why the transform version replaced it.

Modern alternatives

```
/* flexbox */
.parent { display: flex; justify-content: center; align-items: center; min-height: 100vh; }

/* grid, the shortest form of all */
.parent { display: grid; place-items: center; min-height: 100vh; }

/* absolute with inset and automatic margins */
.child { position: absolute; inset: 0; margin: auto; width: 200px; height: 100px; }
```

WHICH ONE TO CHOOSE

For ordinary layout, use grid or flexbox, since the element stays in the flow and the parent keeps its height. Reserve the position and transform technique for elements that must genuinely sit on top of others, such as a modal, a tooltip or a loading spinner over a card.

Points to watch

- The parent must be `position: relative`, or the child will centre itself against the page instead.
- A transform on the child creates a stacking context and also becomes the containing block for any fixed descendants inside it.
- Writing a second `transform` later in the stylesheet overwrites the first, so a hover scale must be written as `transform: translate(-50%, -50%) scale(1.05)`.
- Half-pixel values can make text look slightly blurred, which `translate3d` or a whole-pixel layout usually resolves.

QUICK RECAP

- Offsets move the corner; translate moves the element by its own size.
- The technique works without knowing the dimensions of the box.
- place-items: center is the simplest option when the element can stay in the flow.

17 CSS Grid: Rows and Columns

Defining tracks · The fr unit · Placing and spanning items

Creating a grid

```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-rows: auto 1fr auto;
  gap: 20px;
}
```

Grid is two-dimensional: rows and columns are controlled at the same time, which is the fundamental difference from flexbox, where one axis is managed at a time.

The fr unit

One `fr` represents one share of the **remaining** space, after fixed sizes and gaps have been subtracted. In the example above the two fixed columns and both gaps are removed first, and the middle column then receives everything that is left. This is why `1fr` is safer than `33.33%`, which does not account for the gaps and therefore overflows.

repeat, minmax, auto-fit

```
grid-template-columns: repeat(3, 1fr); /* three equal columns */
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* responsive, no media queries */
```

Read the second line as: fit as many columns as possible, where each is at least two hundred and fifty pixels wide and shares the leftover space equally. The number of columns then changes with the width of the container on its own.

`auto-fill` behaves almost identically but keeps empty tracks in place, so a single item stays at its minimum width instead of stretching across the row. When one item should fill the row, `auto-fit` is the value to choose.

Placing items

```
.item {
  grid-column: 1 / 3; /* from line 1 to line 3, so two columns wide */
  grid-row: 2 / 4;
}

.item { grid-column: span 2; } /* span two columns from wherever it lands */
.item { grid-column: 1 / -1; } /* full width, from the first line to the last */
```

COUNT LINES, NOT COLUMNS

Grid positions are expressed in terms of the lines between tracks. Three columns have four lines. This is why `1 / 3` covers two columns rather than three, and why `-1` is such a convenient way to reach the final line.

Named areas

```
.layout {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar main"
    "sidebar footer";
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
```

Named areas make a page layout readable at a glance, and rearranging the layout for mobile requires only a new set of area strings inside a media query.

Implicit tracks

Items that do not fit into the declared tracks create implicit ones, whose size is controlled by `grid-auto-rows` and `grid-auto-columns`. Setting `grid-auto-rows: minmax(150px, auto)` gives every extra row a sensible minimum height while still allowing it to grow.

QUICK RECAP

- Grid controls both axes at once; fr distributes the space that remains after gaps.
- repeat with auto-fit and minmax produces a responsive grid without breakpoints.
- Placement is based on line numbers, and 1 / -1 spans the full width.

18 CSS Units

px and percentages · rem and em · vh, vw, vmin and vmax

Absolute and relative units

Unit	Relative to	Best used for
px	Nothing; it is fixed	Borders, small radii and fine detail that should not scale.
%	The containing block	Fluid widths inside a parent.
rem	The root font size, normally 16px	Font sizes and spacing scales, because it respects the user's browser setting.
em	The font size of the element itself	Padding inside a component that should scale with its own text.
vw, vh	One per cent of the viewport width or height	Full-screen sections and hero areas.
vmin	One per cent of the smaller viewport dimension	Elements that must stay fully visible in both orientations.
vmax	One per cent of the larger viewport dimension	Decorative shapes that should scale with the longer side.

rem against em

```
html { font-size: 16px; }

.card { font-size: 1.5rem; } /* always 24px */
.card_text { font-size: 1.5em; } /* 1.5 times the parent, so it compounds */
```

The critical difference is **compounding**. Nested elements using `em` multiply: a list inside a list inside a list, each set to 1.2em, grows to roughly 1.7 times the original size. With `rem` the reference is always the root, so the result is predictable no matter how deeply the element is nested.

A PRACTICAL CONVENTION

Use `rem` for type and layout spacing, and `em` for padding that should follow the text size of the component, such as a button whose padding is written as `0.6em 1.2em`. Never set the root font size in pixels in a way that overrides the user's preference; leaving it at the default is the accessible choice.

vmin and vmax in practice

```
.badge { width: 20vmin; height: 20vmin; } /* a circle that never overflows either edge */
.blob { width: 40vmax; } /* scales with the longer side of the screen */
```

On a screen measuring 1200 by 800 pixels, `1vmin` equals eight pixels and `1vmax` equals twelve. Rotate the device and the two swap over, which is exactly what makes `vmin` reliable for elements that must remain fully visible.

Viewport units on mobile

`100vh` is computed against the largest possible viewport, so mobile browser toolbars cause a section to extend below the fold and produce unexpected scrolling. The dynamic units `dvh`, `svh` and `lvh` were introduced for this reason, and `100dvh` is now the sensible default for a full-height section.

FONT SIZES IN VIEWPORT UNITS

Writing `font-size: 4vw` makes text unreadable on a narrow screen and enormous on a wide monitor. Use `clamp()` instead, which is covered in the next topic.

QUICK RECAP

- `rem` refers to the root and does not compound; `em` refers to the element and does.
- `vmin` follows the smaller viewport dimension, `vmax` the larger one.
- Prefer `dvh` over `vh` for full-height sections on mobile devices.

19 CSS Functions: clamp, min, max and var

Fluid sizing without media queries · Custom properties

min() and max()

```
width: min(600px, 90%); /* whichever is SMALLER -> an upper limit */
width: max(300px, 50%); /* whichever is LARGER -> a lower limit  */
```

The naming is the opposite of what intuition suggests. `min()` picks the smaller of the values, which means it acts as a **maximum** on the rendered size. On a wide screen ninety per cent is larger than six hundred pixels, so the width stops at six hundred; on a narrow screen ninety per cent is smaller and therefore wins, leaving a comfortable margin at the sides. A single declaration has replaced a width, a maximum width and a media query.

clamp()

```
font-size: clamp(1.5rem, 4vw, 3rem);
/*          minimum | preferred | maximum */
```

The preferred value is normally expressed in a viewport unit so that it scales continuously with the screen, while the two limits keep it readable at both extremes. This is the standard modern approach to fluid typography, and it removes several breakpoints from a stylesheet.

```
.section { padding-block: clamp(2rem, 6vw, 6rem); }
.container { width: clamp(320px, 90%, 1200px); }
```

KEEP ZOOMING USABLE

A preferred value expressed purely in `vw` ignores browser zoom for text. Combining a relative unit with a viewport unit, as in `clamp(1rem, 0.5rem + 1.5vw, 2rem)`, keeps the text responsive to both the screen and the user's own settings.

Custom properties and var()

```
:root {
  --brand: #1f5f9e;
  --space: 16px;
  --radius: 8px;
}

.card {
  background: var(--brand);
  padding: var(--space);
  border-radius: var(--radius);
}

.card--tight { --space: 8px; } /* the variable is overridden for this component only */
```

- Custom properties are **inherited**, which is what allows a value declared on a parent to be overridden for one subtree.
- A fallback may be supplied as a second argument: `var(--brand, #333)`.
- They are read at run time, so JavaScript can change a value and every rule that uses it updates instantly. This is how theme switching is normally implemented.
- The names are case-sensitive and must begin with two hyphens.

Bringing them together

```
:root { --measure: clamp(320px, 90%, 1100px); }
.wrapper { width: var(--measure); margin-inline: auto; }
```

Functions may also be nested and used in calculations, for example `width: calc(100% - var(--space) * 2)`. Note that `calc()` requires spaces around the plus and minus operators.

QUICK RECAP

- `min()` sets an upper limit and `max()` sets a lower one.
- `clamp()` takes a minimum, a preferred value and a maximum, and replaces breakpoints.
- Custom properties inherit, accept fallbacks and can be changed at run time.